

COMMUNAUTÉ FRANÇAISE
HAUTE ÉCOLE ROBERT SCHUMAN
DÉPARTEMENT ÉCONOMIQUE
6800 LIBRAMONT

Rapport : Équipement d'un personnage de jeu vidéo

Projet présenté pour le cours de
Langages et développement 3 - Labo C++
Bachelier en Informatique

ANDRÉ Allan
DECEMBRE 2025

Je soussigné·e, ...ANDRE Allan....., déclare sur l'honneur que cette production (travail / TFE mémoire / rapport de stage /.....)[1] est entièrement personnelle, fruit de ma réflexion, de mon travail et de mon écriture.

Je certifie que toutes les idées que j'ai pu avancer qui proviennent d'ouvrages, d'articles, de mémoires et/ou de sources sont référencées selon les normes en vigueur. Toutes les citations textuelles sont en outre signalées dans le respect des normes typographiques de référencement.

Je certifie également que :

- ☐ je n'ai en aucune manière utilisé d'IA



[2]

Table des matières

Introduction 3

1. Choix d'implémentation 4

a. Diagramme 4

b. Classe Hero 5

c. Classe Hand 6

d. Classe Object 6

e. Programme principal 6

2. Fonctionnement 7

3. Difficultés rencontrées 13

4. Lignes de commande 14

Conclusion 15

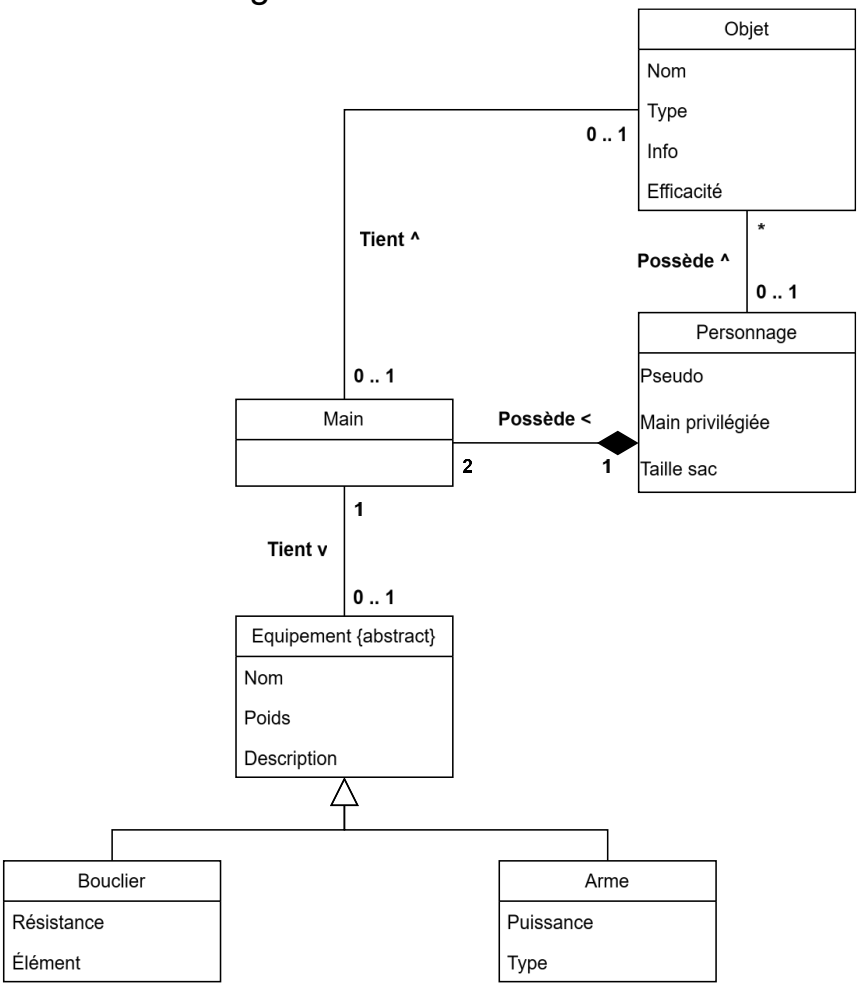
Annexes 16

Introduction

Gérez un équipement dans un jeu vidéo, ce n’est pas toujours simple. Il y a des armes, des boucliers, des objets, et bien d’autres. Idéal pour apprendre à utiliser correctement un Paradigme Orienté Objet. Le but sera donc de coder un programme en C++ permettant à l’utilisateur de créer un équipement pour un personnage de jeu vidéo. Comme je n'essaie pas d’obtenir le GOTY, je me contenterai de permettre à l'utilisateur de gérer un inventaire ainsi que d’équiper des armes, boucliers ou objets.

1. Choix d'implémentation

a. Diagramme



Ma classe Personnage contient une variable indiquant quel est la main principale pour faciliter le changement dans le cas où l'utilisateur souhaiterait changer de main principale. Ainsi, si le héro est gaucher, mais souhaite devenir droitier, il n'y aura qu'une variable à modifier.

Le lien entre la classe Main et la classe Personnage est une composition, car une main ne peut appartenir qu'à une seule personne, et que si la personne disparaît, ses mains disparaissent aussi.

J'ai fait en sorte que Bouclier et Arme héritaient d'Équipement, car ces deux classes avaient des données semblables, et qu'il était dès lors plus simple de les regrouper en une seule classe.

Équipement est abstract car on ne peut pas créer d'équipement seul, il faut que ce soit un Bouclier ou une Arme.

Une main peut tenir soit 1 Objet, soit 1 Équipement, soit rien du tout, mais jamais les deux à la fois.

Un Objet sera soit tenu dans la main du personnage, soit rangé dans son inventaire, mais il ne pourra pas être aux deux endroits à la fois.

b. Classe Hero

Si l'utilisateur a un type d'équipement d'arme dans les deux mains, il peut échanger les armes des mains. Cela permet à l'utilisateur d'échanger facilement ses deux armes plutôt que de devoir les lâcher pour les recréer après.

Quand un utilisateur prend un objet de son inventaire pour l'équiper dans sa main secondaire (sous-entendu que son type d'équipement le lui permet), cet objet est supprimé de son inventaire, car puisqu'il l'a en main, il ne peut pas être ailleurs.

L'utilisateur peur ranger dans son inventaire l'objet qu'il tient en main, autrement, il aurait été obligé de le jeter.

Lorsque l'utilisateur veut équiper un objet, mais qu'il tient déjà un objet dans sa main secondaire, l'objet tenu va automatiquement se ranger dans l'inventaire, permettant à l'utilisateur de l'équiper à nouveau plus tard s'il le désire.

J'ai surchargé l'opérateur [] pour faciliter la récupération d'un objet de l'inventaire.

La fonction add_object() est présente deux fois, une fois en demandant une variable de type Shield, une fois en demandant une variable de type générique. Si la fonction de type générique n'est pas présente, le programme refuse de compiler, car certaines autres fonctions, comme store_object(), appelle la fonction add_object() en lui passant en paramètre une variable dont le compilateur n'est pas sûr que le type est Object, bien que j'ai fait les vérifications préalables.

Toutes les classes surchargent l'opérateur << pour faciliter l'affichage à l'écran, et pour ne pas avoir des variables de même type avec un affichage différent.

c. Classe Hand

Cette classe, ainsi que les classes Object, Weapon et Shield, possèdent une fonction init_empty(). Cette fonction permet de réinitialiser le contenu des variables, ce qui est utile lorsque l'utilisateur veut libérer une main par exemple, car cela la remet dans son état d'origine.

La fonction is_empty(), aussi présente dans les classes citées au paragraphe précèdent, indique si la variable est vide, c'est-à-dire si elle est dans son état d'origine. Pratique pour éviter d'équiper un équipement vide.

La fonction set_holding() est présente deux fois pour la même raison que la fonction add_object() de la classe Hero. Idem pour la surcharge de l'opérateur =.

d. Classe Object

La surcharge de l'opérateur == permet de comparer plus efficacement un string avec le nom de l'objet, utile lorsque l'on cherche à équiper un objet de l'inventaire depuis son nom. Cela est plus efficace que de devoir récupérer le nom de l'objet avec get_name() avant de faire la comparaison.

e. Programme principal

Lors de l'interaction avec l'utilisateur dans le menu, le programme attend un char. Le but est de rendre le programme plus résistant, permettant à l'utilisateur de rentrer tout ce qu'il lui passe par la tête sans faire crasher le programme, tout en limitant l'espace mémoire utilisé.

Toutes les fonctions ask_*() dans les fichiers keyboard.* permettent de récupérer une information auprès de l'utilisateur, tout en atteignant une granularité acceptable.

2. Fonctionnement

Mon programme commence par demander à l'utilisateur de créer son héro.

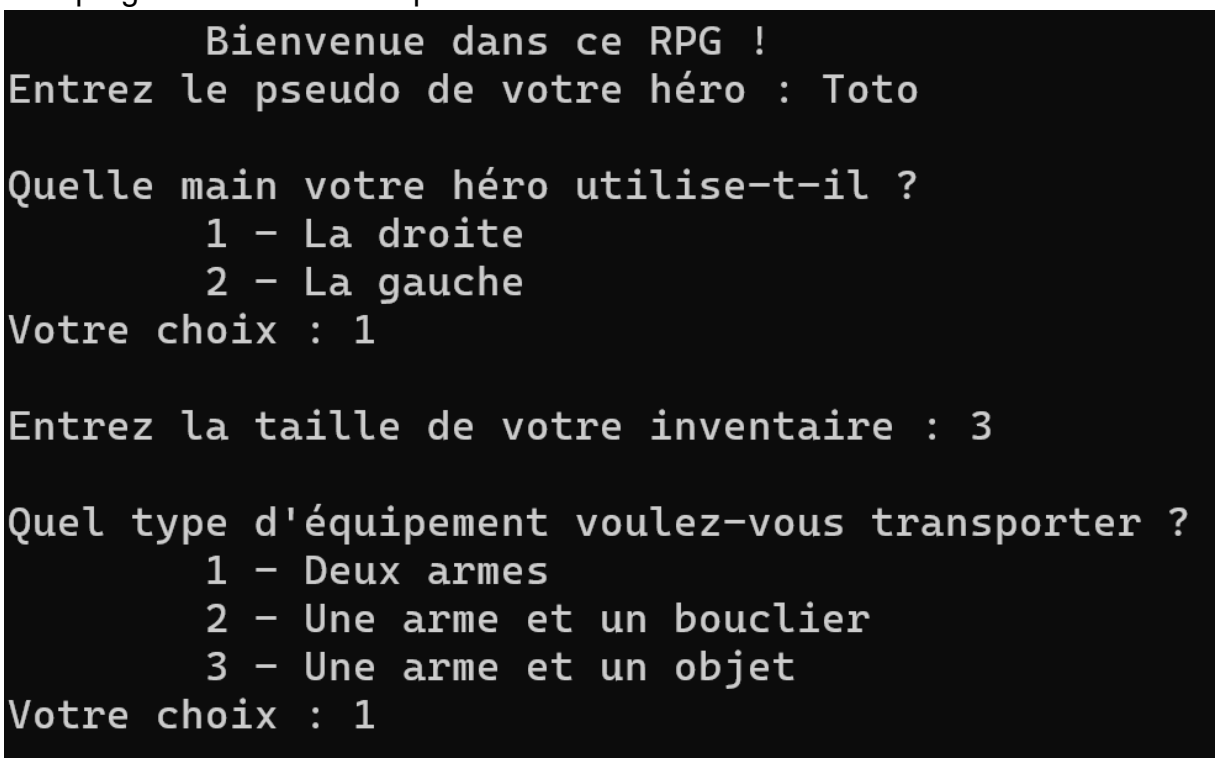


Fig 1 : Création du héro

La création consiste à donner un nom au héro, indiquer sa main principale et son type d'équipement, ainsi que de définir la taille de son inventaire. Le programme ouvre ensuite le menu :

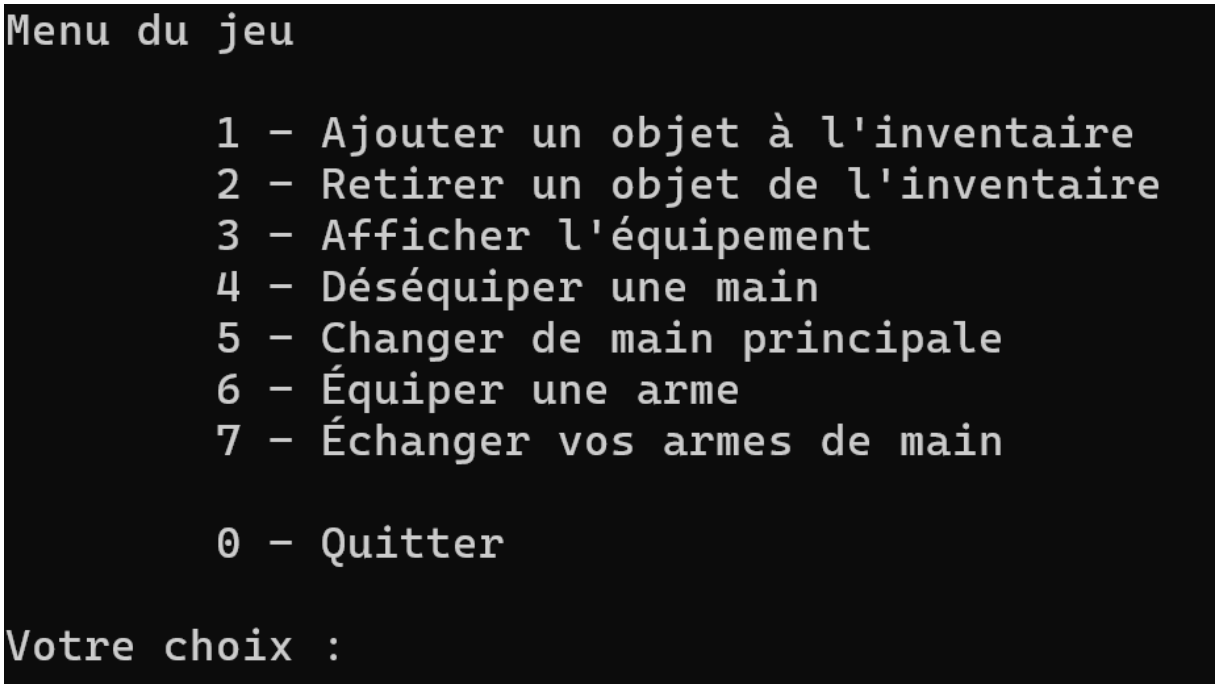


Fig 2 : Menu pour un équipement d'armes dans les deux mains

Ce menu est spécifique aux équipements d'armes dans les deux mains. Vous pouvez retrouver les autres types de menu dans les annexes (fig.11 & fig.12).

Affichons d'abord l'équipement (choix 3) :

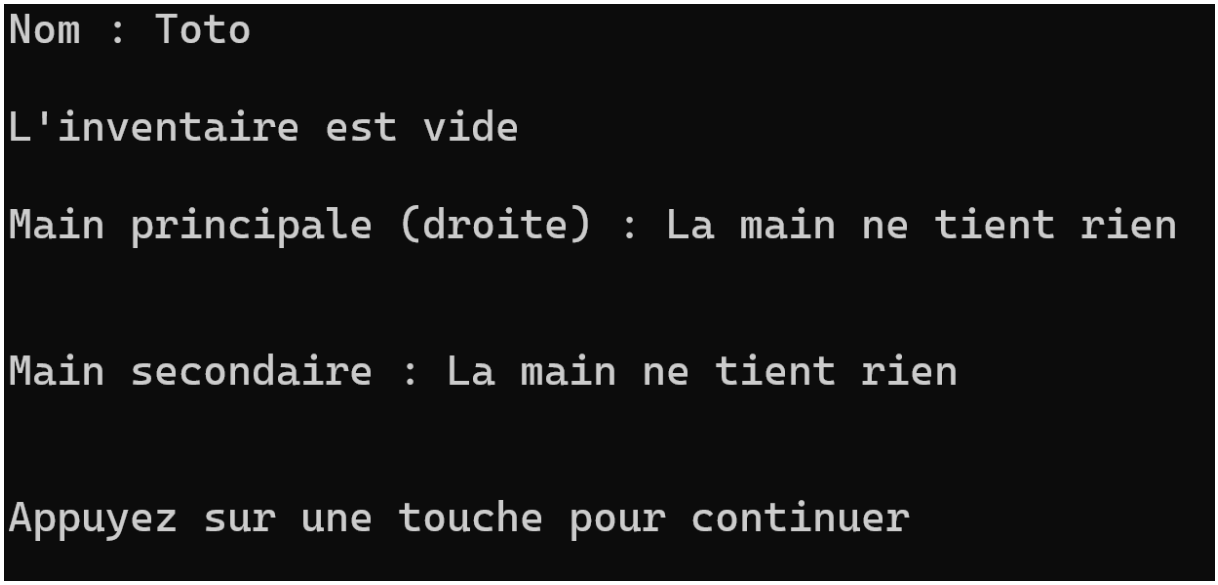


Fig 3 : Équipement vide

On peut constater qu'à part le nom, tout le reste est vide. Remédions à cela, en commençant par créer un inventaire (choix 1) :

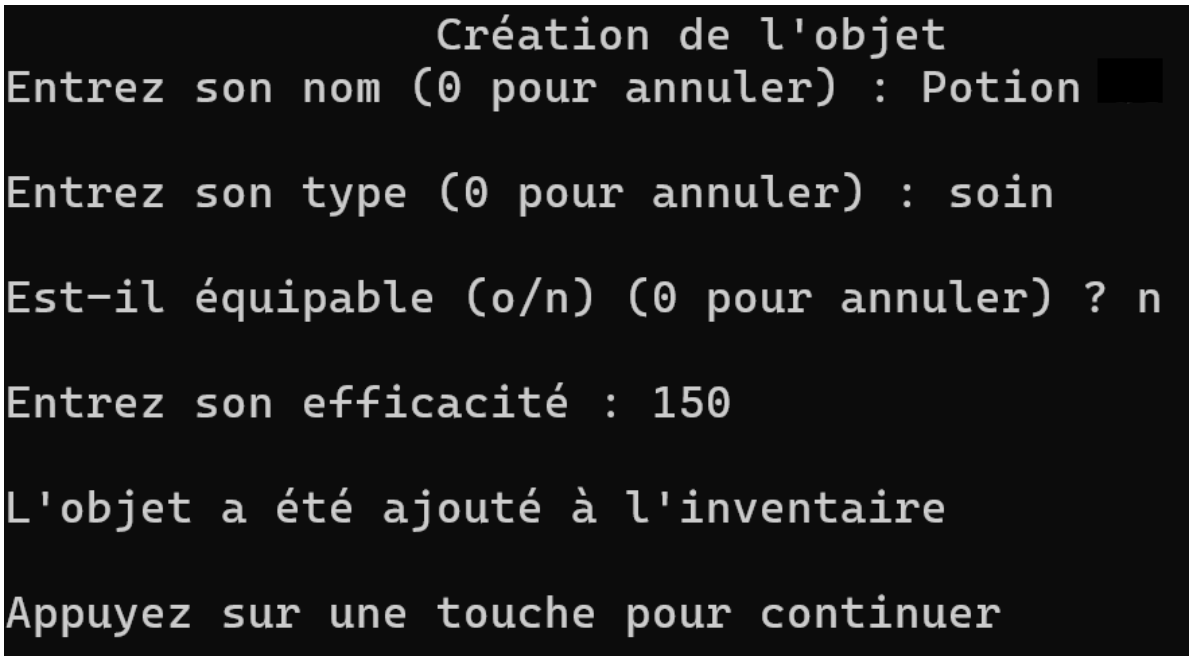


Fig 4 : Création d'un objet

Dans notre cas, puisque lors de la création du héro nous n'avons pas choisi de pouvoir équiper des objets, indiquez si un objet est équipable ou non est inutile.

Équipons maintenant des armes :

```
               Création de l'arme
Entrez son nom (0 pour annuler) : Excalibur

Entrez son poids (-1 pour annuler) : 12.54

Entrez sa description (0 pour annuler) : Épée du légendaire roi Arthur

Entrez son type (0 pour annuler) : épée

Entrez sa puissance : 25

Appuyez sur une touche pour continuer
```

Fig 5 : Création d'une arme

Et voici notre équipement au complet :

```
Nom : Toto

Inventaire :
1) Nom : Potion
   Type : soin
   L'objet ne peut pas être équipé
   Efficacité : 150

2) Nom : Loupe magique
   Type : Révèle les pièges
   L'objet peut être équipé
   Efficacité : 60

3) Nom : Potion
   Type : poison
   L'objet ne peut pas être équipé
   Efficacité : -100


Main principale (droite) :
   Nom : Excalibur
   Poids : 12.54
   Description : Épée du légendaire roi Arthur
   Type : épée
   Puissance : 25

Main secondaire :
   Nom : Baguette de Merlin
   Poids : 9.24
   Description : Baguette magique de Merlin l'Enchanteur
   Type : baguette
   Puissance : 34

Appuyez sur une touche pour continuer
```

Fig 6 : Équipement complet

Imaginons maintenant vouloir ajouter un objet dans notre inventaire. Malheureusement, le programme indique que notre sac est plein. Il va donc falloir libérer un espace (choix 2) :

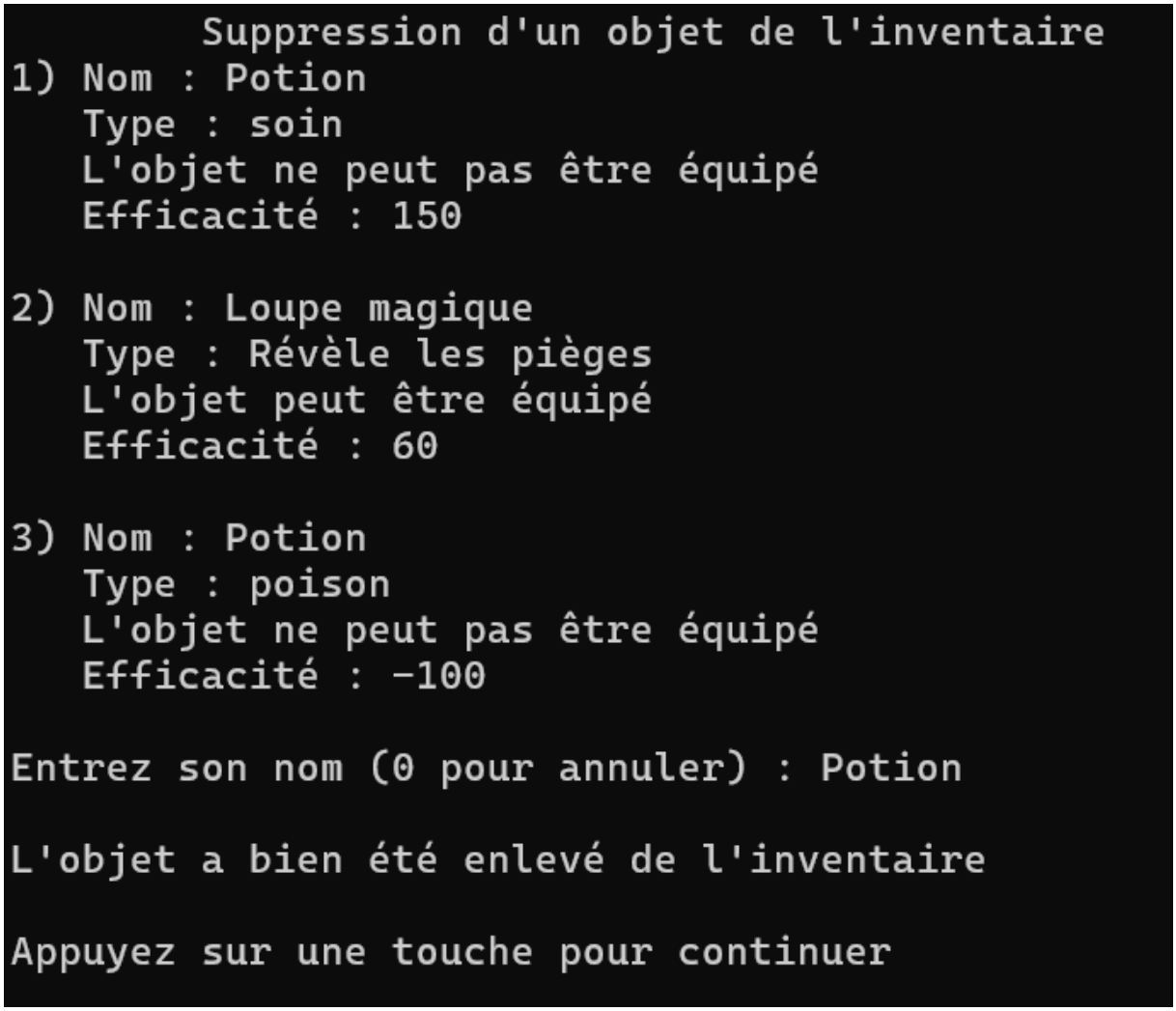


Fig 7 : Suppression d'un objet de l'inventaire

Le programme va supprimer de l'inventaire le premier objet possédant le nom « Potion », ici la potion de soin à l'emplacement 1.

Et si on veut lâcher une arme ? Il suffit d'indiquer la main que l'on veut libérer (choix 4) :

```
Main principale :
   Nom : Excalibur
   Poids : 12.54
   Description : Épée du légendaire roi Arthur
   Type : épée
   Puissance : 25

Main secondaire :
   Nom : Baguette de Merlin
   Poids : 9.24
   Description : Baguette magique de Merlin l'Enchanteur
   Type : baguette
   Puissance : 34

Quelle main voulez-vous déséquiper ?

        1 - La main principale
        2 - La main secondaire

        0 - Quitter

Votre choix : 1
La main principale est désormais vide ou tient l'arme précédemment tenue par la main secondaire

Appuyez sur une touche pour continuer
```

Fig 8 : Déséquiper la main principale

Nous avons lâché l'épée Excalibur, mais comme notre héro la tenait dans sa main principale et qu'on avait choisi qu'il puisse équiper deux armes, il va désormais tenir la baguette de Merlin à la place, car il est plus précis avec sa main principale qu'avec sa main secondaire.

Si on avait voulu équiper un bouclier, il aurait fallu l'indiquer au début lors de la création du personnage. Ensuite, la création d'un bouclier est fort semblable à la création d'une arme :

```
                Création du bouclier
Entrez son nom (0 pour annuler) : Medusa

Entrez son poids (-1 pour annuler) : 8.45

Entrez sa description (0 pour annuler) : Bouclier à l'effigie de Médusa

Quel sera son élémént ?
        1 - Feu
        2 - Air
        3 - Eau
        4 - Terre

        0 - Annuler
Votre choix : 4

Entrez sa résistance : 87

Appuyez sur une touche pour continuer
```

Fig 9 : Création d'un bouclier

On pouvait aussi choisir d'avoir un objet plutôt qu'une deuxième arme ou un bouclier. Dans ce cas-là, pour l'équiper, il faut d'abord posséder l'objet dans l'inventaire du héro, et que cet objet soit équipable.

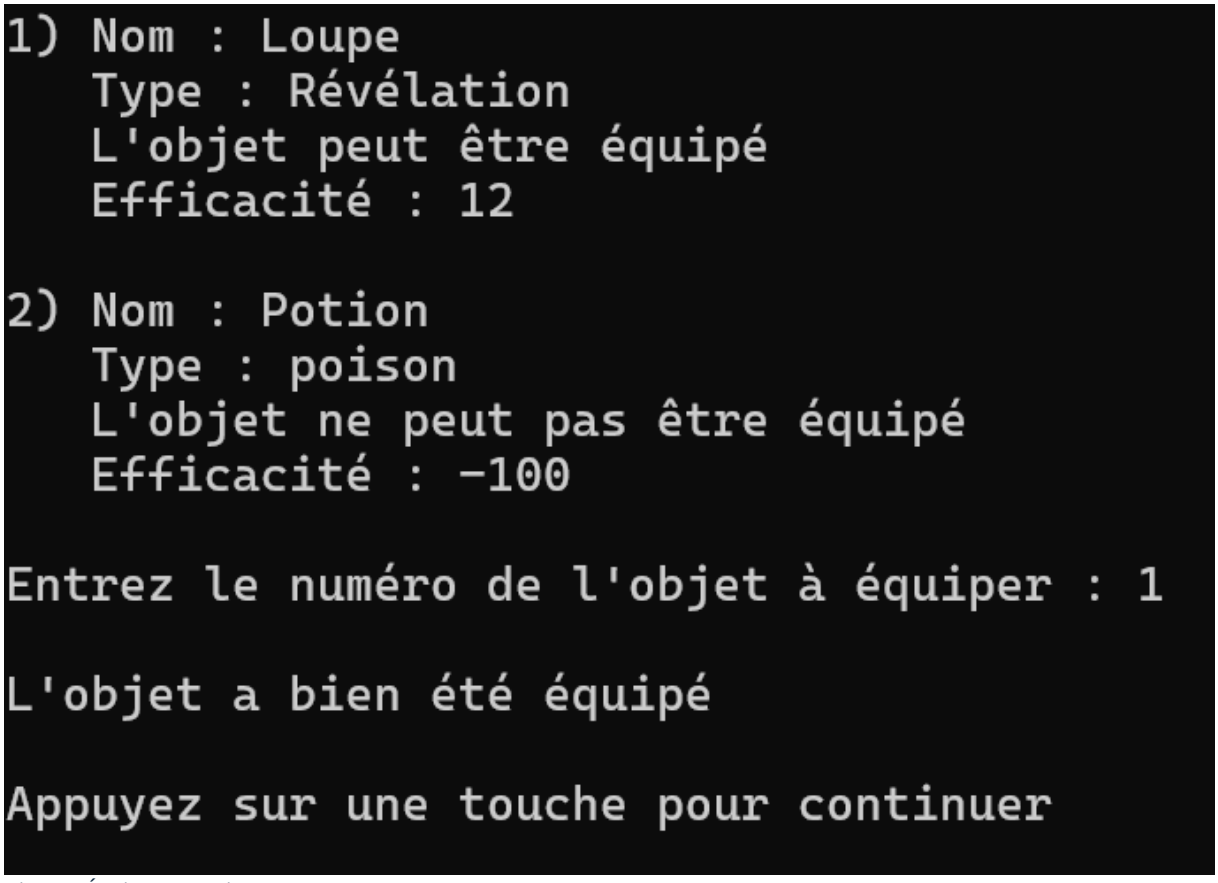


Fig 10 : Équiper un objet

3. Difficultés rencontrées

Lorsque le joueur essayait d'équiper un équipement du mauvais type par rapport au type d'équipement choisi au début de la partie, le programme plantait. Ce bug était dû au fait que mes fonctions carry prenait le type recherché (donc la fonction carry_object attendait une variable de type Object). Pour régler ce problème, je les ai modifiées pour utiliser des types génériques.

J'avais commencé par faire de la classe Hero une classe paramétrable pour la taille de l'inventaire qui aurait été un tableau, mais comme c'était à l'utilisateur de rentrer la taille de son inventaire, ce paramètre n'aurait pas été constant. J'ai donc changé pour faire de size_bag une variable de ma classe Hero et fait de l'inventaire un pointeur d'objet.

Quand le programme demandait un caractère à l'utilisateur et que celui-ci entrait une chaîne de caractère, le programme lançait une boucle infinie. Le problème venait du fait que je n'avais pas mis de fonction pour vider le cache, car je pensais qu'endl, en plus de faire un saut de ligne, vidait automatiquement le cache en même temps.

J'avais parfois un problème de Segmentation Fault, mais j'ai fini par me rendre compte que ça arrivait parce que je n'avais pas de constructeur de copie pour ma classe Hero, alors quand je passais ma variable en paramètre d'une fonction, le programme faisait une copie automatique, ce qui gardait l'adresse de inventory puisque cette variable est un pointeur. Arriver à la fin de la fonction, le programme appelait le destructeur de Hero qui libérait l'espace mémoire utilisé par inventory, ce qui rendait impossible l'accès aux données contenu par ce pointeur.

Je n'arrive pas à séparer certaines fonctions du fichier où elles se trouvent actuellement. Ces fonctions sont celles dans les fichiers Hero.hpp et Hand.hpp nécessitant un autre template, et les fonctions présentes dans le fichier rpg.cpp qui reçoivent une variable de type Hero<T> en paramètre.

4. Lignes de commande

cd src/

g++ -Wall -Werror -Wshadow classes/Object.hpp classes/Object.cpp classes/Equipment.hpp classes/Equipment.cpp classes/Weapon.hpp classes/Weapon.cpp classes/Shield.hpp classes/Shield.cpp classes/Hand.hpp classes/Hand.cpp classes/Hero.hpp classes/Hero.cpp lib/keyboard.hpp lib/keyboard.cpp lib/display.hpp lib/display.cpp rpg.cpp -o play

./play

Conclusion

Ça n’a pas été des plus simples, mais j’ai réussi à coder un programme permettant de gérer l’équipement d’un personnage de jeu vidéo de manière basique, tout en utilisant les concepts vus lors des cours. Le code n’est évidemment pas parfait, notamment en ce qui concerne la séparation en fichiers, mais je suis satisfait du produit fini.

Annexes

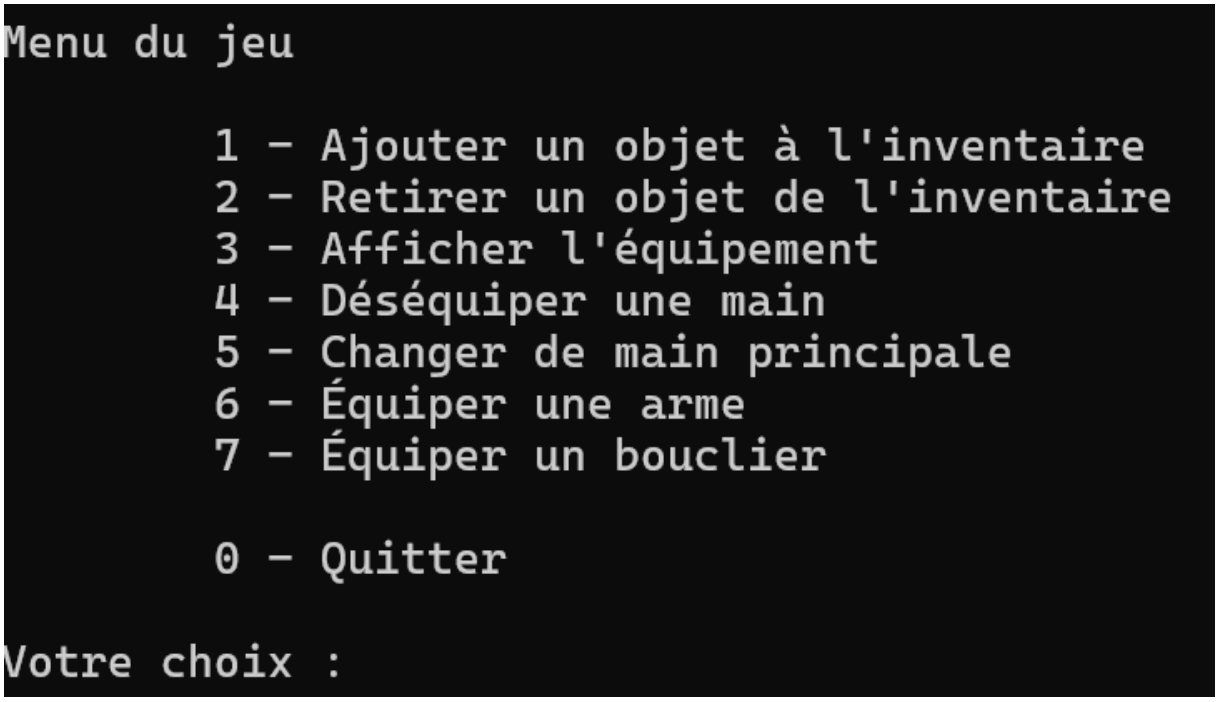


Fig 11 : Menu pour un équipement d'une arme et d'un bouclier

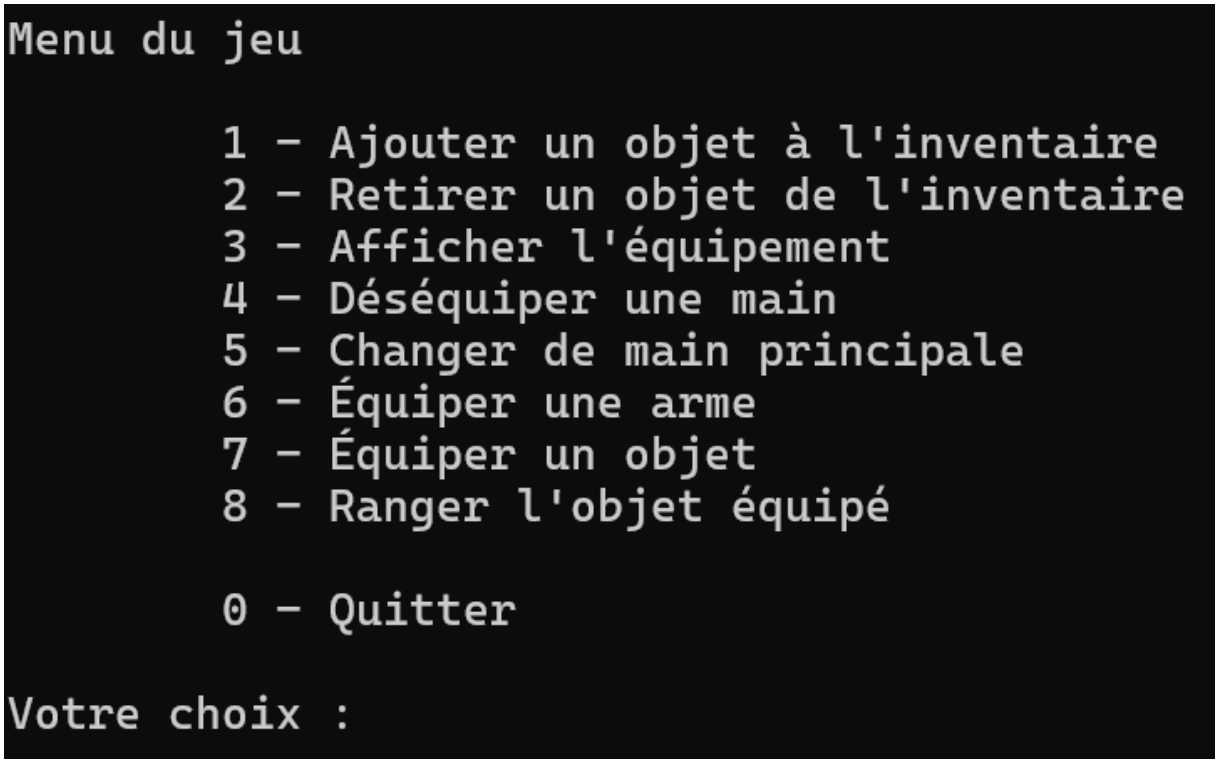


Fig 12 : Menu pour un équipement d'une arme et d'un objet